Guía de comandos Git para Azure DevOps con ramas protegidasRepositorio de ejemplo (placeholder): https://dev.azure.com/MiOrganizacion/MiProyecto/_git/MiRepoRamas exactas: main, develop, feature/oracle-sdlc, hotfix/oracle-patch.Usamos principalmente git switch (disponible desde Git ≥ 2.23). Es más claro e intencional que git checkout (que sirve tanto para cambiar de rama como para restaurar archivos). Si tu Git es más antiguo, sustituye git switch por git checkout.

---

1. Clonar el repositorio desde Azure DevOps

bash

```
# Clonar el repositorio completo
git clone https://dev.azure.com/MiOrganizacion/MiProyecto/_git/MiRepo

# Entrar en el directorio del repositorio
cd MiRepo

# Verificar las ramas remotas disponibles
git branch -r

# Configurar el tracking de la rama principal (opcional pero recomendado)
git switch main
```

Comando / fragmento	Qué hace	Por qué se usa aquí	Riesgo o advertencia
git clone https://...	Descarga todo el historial del repositorio y crea una copia local	Es el punto de partida obligatorio	Ninguno (pide autenticación de Azure DevOps)
cd MiRepo	Cambia al directorio del repositorio clonado	Para trabajar dentro de él	Ninguno
git branch -r	Lista solo las ramas remotas	Para confirmar que existen origin/main, origin/develop, etc.	Ninguno
git switch main	Cambia a la rama main local (creándola si es necesario y vinculándola)	Deja el working tree en la rama principal	Ninguno

Buena práctica: Clona siempre con HTTPS o SSH según la política de tu organización y configura git config --global credential.helper para no introducir credenciales cada vez.

---

2. Validar si la rama local necesita merge o no (sin modificar el working tree)

bash

```
# Actualizar la información de las ramas remotas (sin traer cambios)
git fetch origin

# Ver el estado corto de la rama actual respecto al remoto
```

```
git status -sb

# Contar cuántos commits estamos detrás del remoto
git rev-list --count HEAD..origin/develop

# Contar cuántos commits estamos por delante del remoto
git rev-list --count origin/develop..HEAD

# Ver la diferencia exacta de commits (opcional)
git log --oneline HEAD..origin/develop
```

Comando / fragmento	Qué hace	Por qué se usa aquí	Riesgo o advertencia
git fetch origin	Descarga solo la información de commits y refs del remoto	Actualiza el conocimiento local sin tocar archivos	Ninguno
git status -sb	Muestra estado corto: rama actual, “ahead N”, “behind N”	Diagnóstico rápido y legible	Ninguno
git rev-list --count HEAD..origin/develop	Cuenta los commits que tiene el remoto y nosotros no	Si > 0 → estamos “behind” → hace falta merge o rebase	Ninguno
git rev-list --count origin/develop..HEAD	Cuenta los commits que tenemos y el remoto no	Si > 0 → estamos “ahead”	Ninguno
git log --oneline HEAD..origin/develop	Lista los commits que nos faltan	Para ver qué se va a incorporar	Ninguno

Interpretación clave:

- behind N → necesitas integrar cambios del remoto (merge o pull).
- ahead N → tienes commits locales que aún no están en el remoto.
- Si ambos son 0 → las ramas están sincronizadas, no hace falta merge.
  git status completo muestra archivos modificados; -sb es más limpio para decidir si mergear.

Buena práctica: Ejecuta siempre git fetch antes de decidir. Nunca hagas git pull a ciegas.

---

3. Hacer merge de forma segura y sin errores (incluyendo resolución de conflictos y abortar)
bash

```
# Estar en la rama destino (ejemplo: develop)
git switch develop

# Actualizar la información remota
git fetch origin

# Merge seguro: solo permite fast-forward
```

```
git merge --ff-only origin/develop

# Si falla el --ff-only, usar merge normal (crea commit de merge)
git merge --no-ff origin/feature/oracle-sdlc

# Si aparecen conflictos:
git status                    # Ver archivos en conflicto (Unmerged paths)
# Editar los archivos, eliminar marcadores <<<<<<< ======= >>>>>>>
git add archivo-conflicto.js  # Marcar como resuelto
git merge --continue          # Finalizar el merge (o git commit)

# Si algo sale mal y quieres cancelar todo el merge:
git merge --abort
```

Diferencia de opciones de merge:

- --ff-only: solo avanza el puntero si no hay divergencia. Más seguro, no crea commit de merge innecesario.
- --no-ff: siempre crea un commit de merge explícito (útil para ver la integración de features en el historial).
- --no-commit: prepara el merge pero no hace el commit (permite revisar antes).

Recomendación: Usa --ff-only para actualizaciones diarias de develop/main. Usa --no-ff cuando integres una feature branch para mantener el historial legible.

Comando / fragmento	Qué hace	Por qué se usa aquí	Riesgo o advertencia
git switch develop	Cambia a la rama destino	El merge se hace sobre la rama actual	Ninguno
git fetch origin	Actualiza refs remotas	Evita mergear información desactualizada	Ninguno
git merge --ff-only ...	Intenta fast-forward puro	Evita merges innecesarios y conflictos	Falla si hay divergencia (es intencional)
git merge --no-ff ...	Crea commit de merge	Documenta la integración de la feature	Puede generar conflictos
git status	Lista archivos con conflictos	Identifica qué hay que resolver	Ninguno
git add archivo...	Marca el archivo como resuelto	Prepara el commit de merge	Asegúrate de haber resuelto bien
git merge --continue	Completa el merge	Finaliza el proceso	Ninguno
git merge --abort	Cancela el merge y restaura el estado anterior	Seguridad cuando los conflictos son complejos	Ninguno (es la salida de emergencia)

Buena práctica: Nunca resuelvas conflictos a ciegas. Usa git diff o un merge tool (git mergetool) y prueba el código antes de git merge --continue.

---

4. Cambiar de una rama a otra (incluyendo rama remota inexistente localmente y volver a la anterior)
bash

```
# Ver la rama actual
git branch

# Cambiar a una rama local existente
git switch develop

# Crear y cambiar a una rama remota que aún no existe localmente
git switch --track origin/feature/oracle-sdlc
# (equivalente más moderno: git switch feature/oracle-sdlc  si Git ≥ 2.23 y existe en origin)

# Volver a la rama anterior
git switch -

# Crear una nueva rama local a partir de la actual
git switch -c hotfix/oracle-patch
```

Comando / fragmento	Qué hace	Por qué se usa aquí	Riesgo o advertencia
git branch	Lista ramas locales y marca la actual con *	Orientación	Ninguno
git switch develop	Cambia de rama (working tree limpio)	Navegación segura	Falla si hay cambios sin commit (usa git stash)
git switch --track origin/...	Crea la rama local y la vincula al remoto	Primer uso de una rama remota	Ninguno
git switch -	Vuelve a la rama anterior	Navegación rápida	Ninguno
git switch -c nueva-rama	Crea y cambia a una nueva rama	Inicio de feature/hotfix	Ninguno

Buena práctica: Antes de cambiar de rama, asegúrate de que git status esté limpio o guarda el trabajo con git stash.

---

5. Hacer merge de una rama específica dentro de otra rama específica(ejemplo: feature/oracle-sdlc → develop)
bash

```
# Ir a la rama destino
git switch develop

# Actualizar develop desde el remoto
```

```
git fetch origin
git merge --ff-only origin/develop

# Mergear la feature (recomendado --no-ff para historial claro)
git merge --no-ff feature/oracle-sdlc

# Si todo está bien, subir el resultado
git push origin develop
```

Comando / fragmento	Qué hace	Por qué se usa aquí	Riesgo o advertencia
git switch develop	Posiciona en la rama destino	El merge se aplica sobre ella	Ninguno
git fetch + merge --ff-only	Sincroniza develop antes	Evita conflictos innecesarios	Ninguno
git merge --no-ff feature/...	Integra la feature creando commit de merge	Documenta la integración	Posibles conflictos
git push origin develop	Publica el merge	Comparte el resultado con el equipo	Rechazo si alguien más empujó (ver escenario 6)

Buena práctica: En Azure DevOps, preferiblemente haz el merge mediante Pull Request. El comando anterior es para casos locales o cuando tienes permisos directos.

---

6. Subir archivos a una rama específica que NO sea main(ejemplo: feature/oracle-sdlc)
bash

```
# Estar en la rama de trabajo
git switch feature/oracle-sdlc

# Añadir archivos y crear commit
git add .
git status
git commit -m "feat: implementar lógica oracle-sdlc"

# Primer push (la rama remota aún no existe)
git push -u origin feature/oracle-sdlc

# Push posterior normal
git push

# Si el push es rechazado por non-fast-forward:
git fetch origin
git status -sb
```

```
# Opción segura: traer cambios y reintentar
git merge --no-ff origin/feature/oracle-sdlc
# o rebase (más limpio): git pull --rebase origin feature/oracle-sdlc
git push

# NUNCA uses --force a menos que sepas exactamente lo que haces.
# Si es imprescindible, usa primero:
git push --force-with-lease origin feature/oracle-sdlc
```

Comando / fragmento	Qué hace	Por qué se usa aquí	Riesgo o advertencia
git add .	Prepara todos los cambios	Incluye archivos nuevos/modificados	Puede añadir archivos no deseados (usa .gitignore)
git commit -m "..."	Crea el commit local	Guarda el trabajo	Ninguno
git push -u origin ...	Publica la rama y configura tracking	Primer push de una rama nueva	Ninguno
git push	Sube commits posteriores	Actualización normal	Rechazo non-fast-forward si el remoto avanzó
git fetch + merge/rebase	Integra cambios remotos	Resuelve el rechazo de forma segura	Posibles conflictos
git push --force-with-lease	Sobrescribe el remoto solo si no ha cambiado desde tu último fetch	Más seguro que --force	Aún puede borrar trabajo de otros si no tienes cuidado
git push --force	Sobrescribe el remoto sin comprobar	—	Alto riesgo: borra commits de compañeros. Evitar en ramas compartidas

Diferencia --force vs --force-with-lease:
--force pisa todo.
--force-with-lease solo pisa si el remoto sigue igual que cuando hiciste el último fetch. Es la opción mínima aceptable.Buena práctica: En ramas de feature, usa siempre Pull Request en Azure DevOps. Evita push directo a develop/main (están protegidas).

CHEAT-SHEET – Flujo de trabajo diario
bash

```
# 1. Clonar (solo la primera vez)
git clone https://dev.azure.com/MiOrganizacion/MiProyecto/_git/MiRepo
cd MiRepo

# 2. Actualizar información
git fetch origin

# 3. Ver estado y si necesitas merge
```

```
git status -sb
git rev-list --count HEAD..origin/develop

# 4. Cambiar de rama
git switch develop
git switch --track origin/feature/oracle-sdlc
git switch -

# 5. Trabajar y subir
git add .
git commit -m "mensaje"
git push -u origin feature/oracle-sdlc   # primer push
git push                                 # siguientes

# 6. Merge seguro
git switch develop
git merge --ff-only origin/develop
git merge --no-ff feature/oracle-sdlc
git push origin develop

# 7. Conflictos
git status
# editar archivos
git add <archivos>
git merge --continue
# o
git merge --abort
```

Errores comunes y cómo evitarlos

1. Push rechazado por non-fast-forward
   Causa: alguien más empujó a la misma rama.
   Solución: git fetch → git merge o git pull --rebase → git push. Nunca --force sin --force-with-lease.
2. Intentar mergear o pushear directamente a main
   Causa: rama protegida en Azure DevOps.
   Solución: trabaja siempre en feature/hotfix y usa Pull Request.
3. Olvidar git fetch antes de merge
   Causa: mergeas información antigua.
   Solución: hábito de git fetch origin + git status -sb.

4. Resolver conflictos mal y hacer commit
   Causa: dejar marcadores <<<<<<< o código roto.
   Solución: git status → editar → probar → git add → git merge --continue. Usa git merge --abort si dudas.
5. Cambiar de rama con cambios sin commit
   Causa: Git se niega o pierdes trabajo.
   Solución: git stash antes de git switch, luego git stash pop.
6. Usar git push --force en rama compartida
   Causa: borra commits de compañeros.
   Solución: solo --force-with-lease y únicamente en ramas de feature personales.

Con esta secuencia y las explicaciones puedes trabajar de forma segura en Azure DevOps respetando las ramas protegidas.